Practical No: 00

Aim: Starting Raspbian OS, Familiarizing with Raspberry Pi Components and interface, connecting to Ethernet, Monitor, USB.

Hardware Required:

1. Raspberry Pi 3B+
2. Ethernet Cable
3. Monitor
4. HDMI to VGA convertor
5. Micro SD card (any class best is class 10)
6. Adaptor with 5v 2A
7. USB mouse
8. USB keyboard

Software Required:

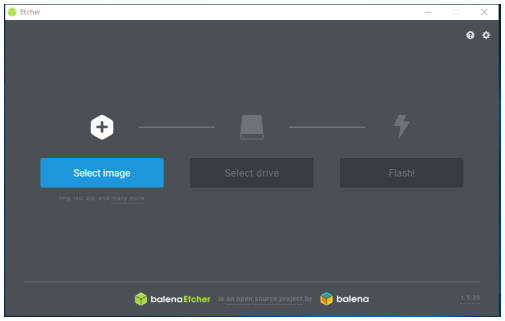
1. Raspbian OS (https://www.raspberrypi.org/downloads/raspbian/)
2. Etcher for Windows (https://www.balena.io/etcher/)

Mr. Anupam

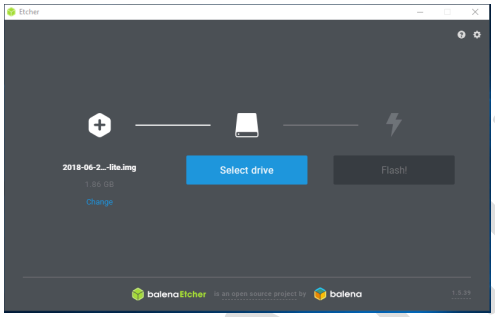
Procedure:

1. Installing Raspbian

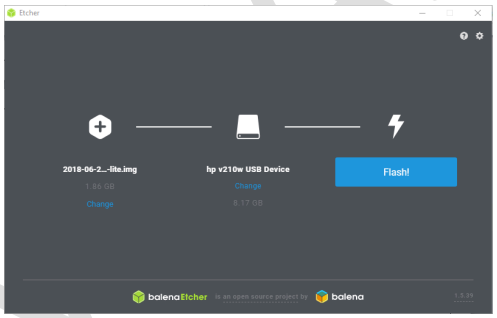
- Download Raspbian OS (will be in zip format).
- Unzip it
- Download the windows installer of Etcher.
- Run Etcher.



- Select the SD card drive.



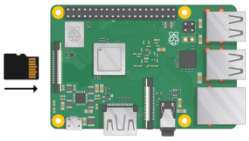
- Finally, click “Flash” to write the Raspbian image to the SD card.



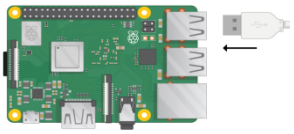
- You’ll see a progress bar. Once complete, the utility will automatically Unmounts the SD card so it is safe to remove it from your computer.

2. Connecting Components

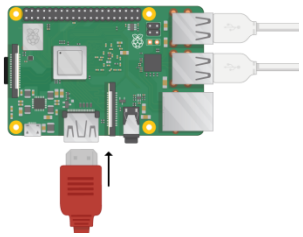
- Insert the prepared SD card in the SD card slot in the Raspberry Pi.



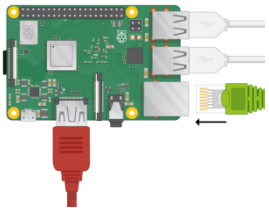
- Connect USB mouse and Keyboard



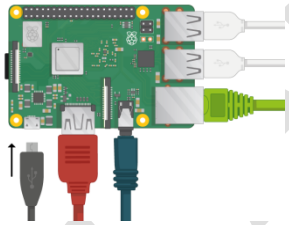
- Connect the HDMI to convertor with the VGI cable with the monitor.



- Connect Ethernet to Router or Switch



- Switch on the monitor
- Provide Power to the Raspberry Pi by connecting Adapter.



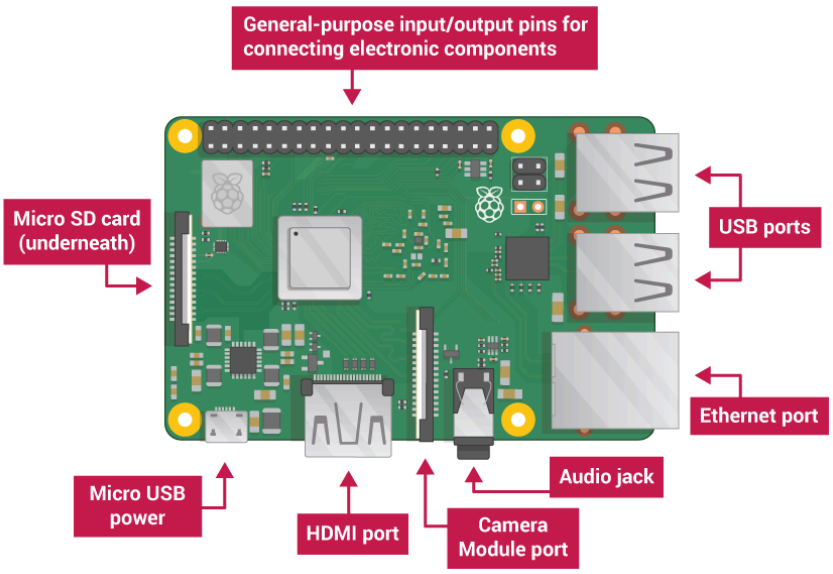
Precautions:

1. While connecting components follow the above steps as it is, remembering that power should be provided at the end.
2. Remember to un-mount the SD card before removing it from PC as this may lead to data corruption.

Conclusion:

1. We have learned the basic steps to get started with Raspberry Pi.

Mr. Anupam



- **USB ports**: these are used to connect a mouse and keyboard. You can also connect other components, such as a USB drive.
- **SD card slot**: you can slot the SD card in here. This is where the operating system software and your files are stored.
- **Ethernet port**: this is used to connect the Raspberry Pi to a network with a cable. The Raspberry Pi can also connect to a network via wireless LAN.
- **Audio jack**: you can connect headphones or speakers here.
- **HDMI port**: this is where you connect the monitor (or projector) that you are using to display the output from the Raspberry Pi. If your monitor has speakers, you can also use them to hear sound.
- **Micro USB power connector**: this is where you connect a power supply. You should always do this last, after you have connected all your other components.
- **GPIO ports**: these allow you to connect electronic components such as LEDs and buttons to the Raspberry Pi.

Practical No: 01

Aim: Displaying different Led Patterns with Raspberry Pi.

Hardware Required:

1. Raspberry Pi 3B+
2. Ethernet Cable
3. Monitor
4. HDMI to VGA convertor
5. Micro SD card (any class best is class 10)
6. Adaptor with 5v 2A
7. USB mouse
8. USB keyboard
9. 3 LED
10. Breadboard
11. Wires

Software Required:

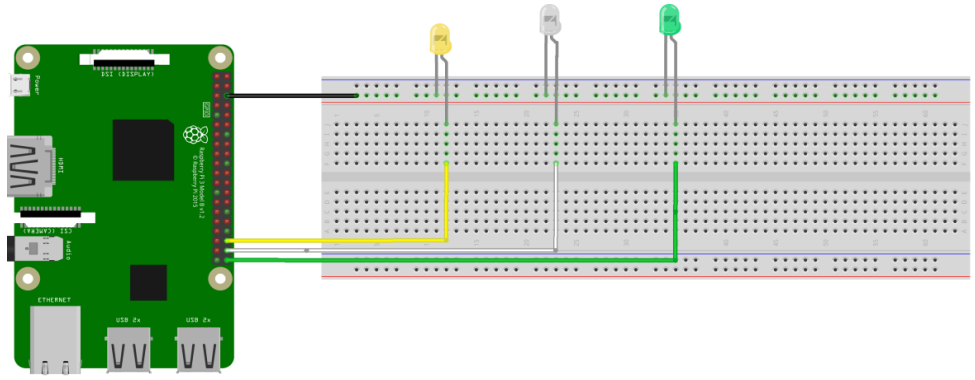
1. Raspbian OS

Mr. Anupam

Procedure:

1. Hardware Setup

- Connect components as in the Diagram



G. [illegible] LED1 Anode

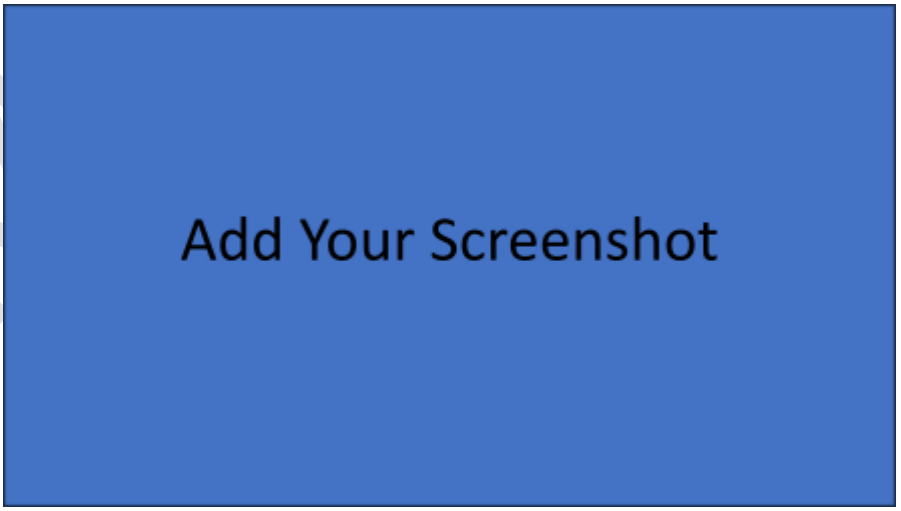
GPIO20 □ LED2 Anode

GPIO21 □ LED3 Anode

*Black wire is the GND (ground)

2. Software Setup

- Geany Programmer's Editor
- Rapberry □Programming □ Geany Programmer's Editor

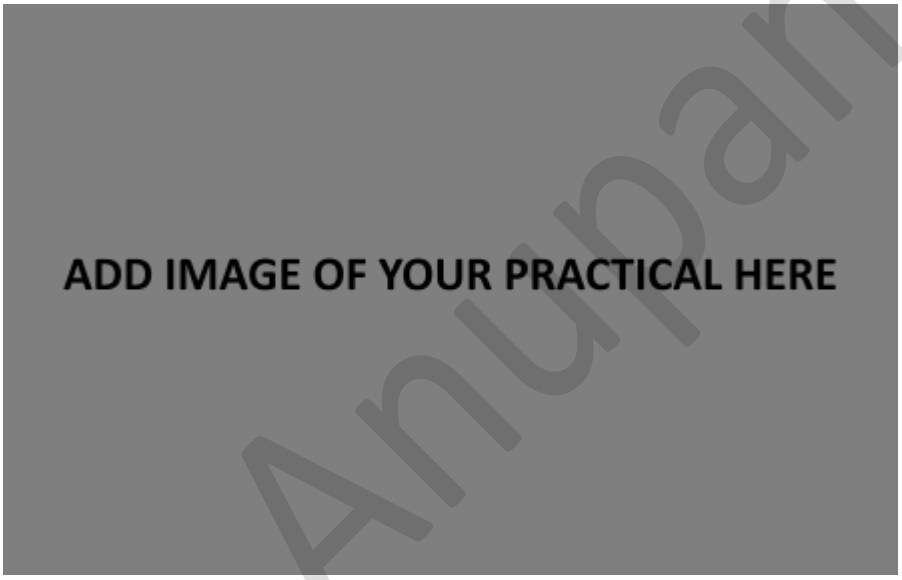


- File □ New
- File □ Save (provide file name anything) *.py
- Type The Following Code

**COPY PASTE YOUR CODE HERE **

Conclusion:

1. In this practical we have learned how to connect GPIO pins to LEDs.
2. We have learned about “gpiozero” and “time” module of Python.



Mr. Anupam

Practical No: 04

Aim: Controlling Raspberry Pi with Telegram.

Hardware Required:

1. Raspberry Pi 3B+
2. Ethernet Cable
3. Monitor
4. HDMI to VGA convertor
5. Micro SD card (any class best is class 10)
6. Adaptor with 5v 2A
7. USB mouse
8. USB keyboard
9. Relay board
10. Female – male jumper wires.
11. Bread Board

Software Required:

1. Raspbian OS
2. Geany Programmer's Editor
3. Telegram App

Mr. Anupam

Procedure:

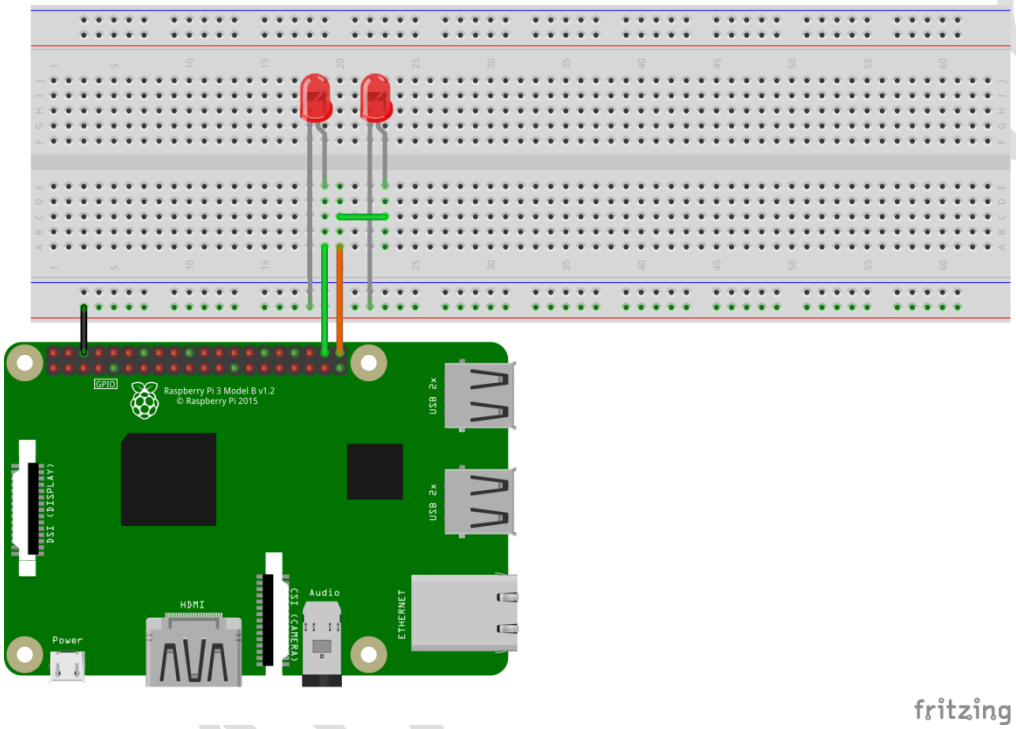
1. **Hardware Setup:**

   - Connect according to the figure.

     GPIO20 □ LED2 Anode

     GPIO21 □ LED3 Anode

     *Black wire is the GND (ground)



2. **Software Setup:**

   - Install Telegram in android phone.
   - Search for contact “BotFather”
   - Send him “/start” command
   - Send him “/newbot”
   - Give a name to you bot
   - Give a username to the bot (It must be a unique one)
   - On success you will be provided with a HTTP API token. (jot it down)
   - In Raspberry Pi Terminal type the following…
     - sudo apt update
     - sudo apt upgrade
     - sudo pip install teleport --break-system-packages

- open Geany Programmer's Editor and create a python script as given below

```
import time
import telepot
from gpiozero import LED

# GPIO Pins (BCM numbering)
red = LED(21)
blue = LED(20)
green = LED(27)

# ----------------------
# LED Functions
# ----------------------

def red_on():
    red.on()
    return "🔴 Red LED is ON"

def red_off():
    red.off()
    return "🔴 Red LED is OFF"

def blue_on():
    blue.on()
    return "🔵 Blue LED is ON"

def blue_off():
    blue.off()
    return "🔵 Blue LED is OFF"

def green_on():
    green.on()
    return "🟢 Green LED is ON"

def green_off():
    green.off()
    return "🟢 Green LED is OFF"
```

Mr. Anupam

```
def all_on():
    red.on()
    blue.on()
    green.on()
    return "💡 All LEDs are ON"


def all_off():
    red.off()
    blue.off()
    green.off()
    return "💡 All LEDs are OFF"


def blink_all():
    for i in range(5):
        red.on()
        blue.on()
        green.on()
        time.sleep(0.5)

        red.off()
        blue.off()
        green.off()
        time.sleep(0.5)

    return "✨ Blink Completed"


def status():
    return (
        f"LED Status\n"
        f"🔴 Red   : {'ON' if red.is_lit else 'OFF'}\n"
        f"🔵 Blue  : {'ON' if blue.is_lit else 'OFF'}\n"
        f"🟢 Green : {'ON' if green.is_lit else 'OFF'}"
    )

def alternate():
    for i in range(10):
        red.on()
        blue.off()
```

Mr. Anupam

```
        green.off()
        time.sleep(0.3)

        red.off()
        blue.on()
        green.off()
        time.sleep(0.3)

        red.off()
        blue.off()
        green.on()
        time.sleep(0.3)

    all_off()
    return "🔄 Alternate Pattern Completed"


def police():
    for i in range(15):
        red.on()
        blue.off()
        time.sleep(0.15)

        red.off()
        blue.on()
        time.sleep(0.15)

    all_off()
    return "🚔 Police Lights Completed"


def traffic():
    all_off()

    red.on()
    time.sleep(2)

    red.off()
    green.on()
    time.sleep(2)
```

Mr. Anupam

```
    green.off()
    blue.on()
    time.sleep(2)

    blue.off()

    return "🚦 Traffic Light Cycle Completed"


def rainbow():
    for i in range(5):
        red.on()
        time.sleep(0.3)

        red.off()
        blue.on()
        time.sleep(0.3)

        blue.off()
        green.on()
        time.sleep(0.3)

        green.off()
        red.on()
        blue.on()
        time.sleep(0.3)

        red.off()
        blue.off()
        green.on()
        time.sleep(0.3)

        green.off()
        red.on()
        green.on()
        time.sleep(0.3)

        all_off()

    return "🌈 Rainbow Pattern Completed"
```

Mr. Anupam

```
def chase():
    for i in range(8):
        red.on()
        time.sleep(0.2)
        red.off()

        blue.on()
        time.sleep(0.2)
        blue.off()

        green.on()
        time.sleep(0.2)
        green.off()

    return "🏃 Chase Pattern Completed"


def flash_red():
    for i in range(10):
        red.toggle()
        time.sleep(0.2)

    red.off()
    return "🔴 Red Flash Completed"


def flash_blue():
    for i in range(10):
        blue.toggle()
        time.sleep(0.2)

    blue.off()
    return "🔵 Blue Flash Completed"


def flash_green():
    for i in range(10):
        green.toggle()
        time.sleep(0.2)
```

Mr. Anupam

```
    green.off()
    return "🟢 Green Flash Completed"


def disco():
    leds = [red, blue, green]

    for i in range(40):
        all_off()
        random.choice(leds).on()
        time.sleep(0.15)

    all_off()
    return "🎉 Disco Mode Completed"


def wave():
    for i in range(5):
        red.on()
        time.sleep(0.2)

        blue.on()
        time.sleep(0.2)

        green.on()
        time.sleep(0.2)

        red.off()
        time.sleep(0.2)

        blue.off()
        time.sleep(0.2)

        green.off()
        time.sleep(0.2)

    return "🌊 Wave Pattern Completed"


def heartbeat():
    for i in range(8):
```

Mr. Anupam

```
        all_on()
        time.sleep(0.1)
        all_off()
        time.sleep(0.1)
        all_on()
        time.sleep(0.3)
        all_off()
        time.sleep(0.5)

    return "❤️ Heartbeat Pattern Completed"


def help_menu():
    return """
Available Commands

🔴 on red
🔴 off red
🔵 on blue
🔵 off blue
🟢 on green
🟢 off green

💡 all on
💡 all off

✨ blink

🚔 police
🚦 traffic
🌈 rainbow
🏃 chase
🔄 alternate
🎉 disco
🌊 wave
❤️ heartbeat

🔴 flash red
🔵 flash blue
🟢 flash green
```

Mr. Anupam

```
📊 status
❓ help
"""

# ----------------------
# Telegram Handler
# ----------------------

def handle(msg):
    chat_id = msg['chat']['id']
    command = msg.get('text', '').lower().strip()

    print("Received:", command)

    if command == "on red":
        bot.sendMessage(chat_id, red_on())

    elif command == "off red":
        bot.sendMessage(chat_id, red_off())

    elif command == "on blue":
        bot.sendMessage(chat_id, blue_on())

    elif command == "off blue":
        bot.sendMessage(chat_id, blue_off())

    elif command == "on green":
        bot.sendMessage(chat_id, green_on())

    elif command == "off green":
        bot.sendMessage(chat_id, green_off())

    elif command == "all on":
        bot.sendMessage(chat_id, all_on())

    elif command == "all off":
        bot.sendMessage(chat_id, all_off())

    elif command == "blink":
        bot.sendMessage(chat_id, blink_all())
```

Mr. Anupam

```
    elif command == "status":
        bot.sendMessage(chat_id, status())

    elif command == "help":
        bot.sendMessage(chat_id, help_menu())
    elif command == "alternate":
      bot.sendMessage(chat_id, alternate())

  elif command == "police":
    bot.sendMessage(chat_id, police())

elif command == "traffic":
    bot.sendMessage(chat_id, traffic())

elif command == "rainbow":
    bot.sendMessage(chat_id, rainbow())

elif command == "chase":
    bot.sendMessage(chat_id, chase())

elif command == "flash red":
    bot.sendMessage(chat_id, flash_red())

elif command == "flash blue":
    bot.sendMessage(chat_id, flash_blue())

elif command == "flash green":
    bot.sendMessage(chat_id, flash_green())

elif command == "disco":
    bot.sendMessage(chat_id, disco())

elif command == "wave":
    bot.sendMessage(chat_id, wave())

elif command == "heartbeat":
    bot.sendMessage(chat_id, heartbeat())

    else:
        bot.sendMessage(
            chat_id,
```

Mr. Anupam

```
            "❌ Unknown Command\n\nType 'help' to see available commands."
        )


# ----------------------
# Telegram Bot
# ----------------------

TOKEN = "YOUR_TELEGRAM_BOT_TOKEN"

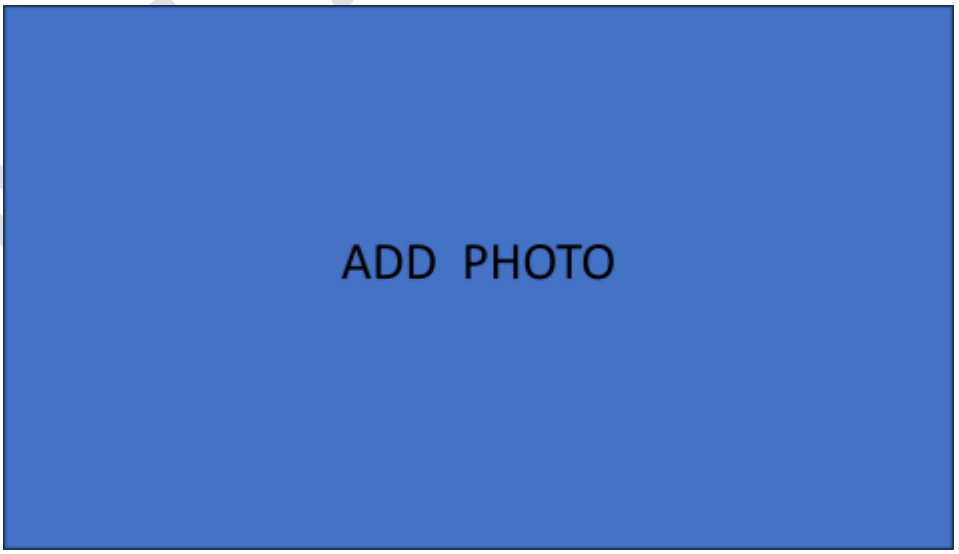
bot = telepot.Bot(TOKEN)

bot.message_loop(handle)

print("🤖 Bot is running...")

try:
    while True:
        time.sleep(10)

except KeyboardInterrupt:
    red.off()
    blue.off()
    green.off()
    print("Program Stopped.")
```



- Run the program by clicking on the rocket Icon.

Anupam

Precautions:

- If it is a fresh flash of Rasbian OS “*sudo apt update”* and “*sudo apt upgrade”* is a must thing.
- Connect the components before powering on the device and double check your connections.
- Take the help of the professor in connection as this project includes use of high voltage.

Conclusion:

- We are able to control two LED lights through Telegram app.
- By sending “on led1” switches on the led1 and “off led1” switches off led1, and vise-versa

Mr. Anupam

Practical No: 03

Aim: Displaying Time over 16x2 LCD Display using Raspberry Pi.

Hardware Required:

1. Raspberry Pi 3B+
2. Ethernet Cable
3. Monitor
4. HDMI to VGI convertor
5. Micro SD card (any class best is class 10)
6. Adaptor with 5v 2A
7. USB mouse
8. USB keyboard
9. jumper wires
10. 16x2 LCD Display
11. Breadboard

Software Required:

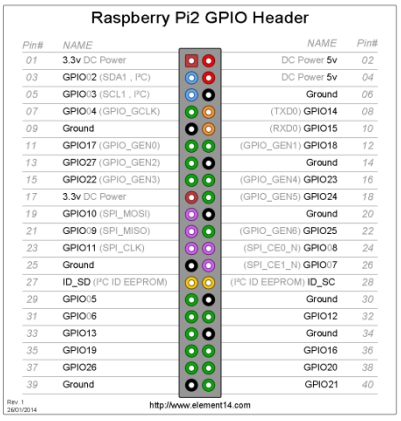
1. Rapbeian OS
2. Thonny Python IDE

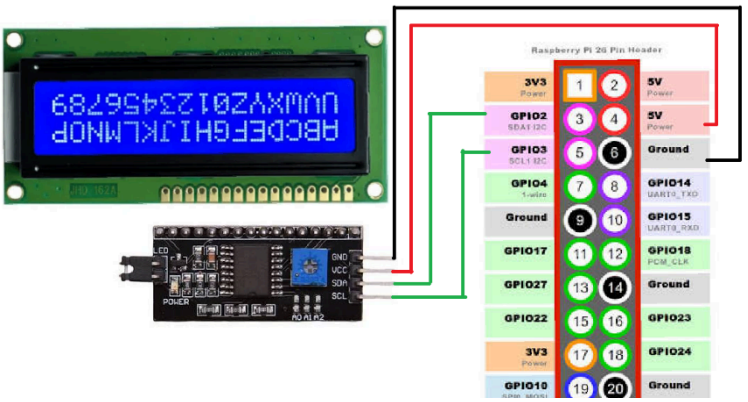
Procedure:

1. Hardware Setup:

- Connect the pins as given bellow….

- **SDA (LCD) → GPIO 2 (Pin 3)** on Pi
- **SCL (LCD) → GPIO 3 (Pin 5)** on Pi
- **VCC → 5V (Pin 2 or 4)**
- **GND → Ground (Pin 6)** Most I2C LCDs use the **PCF8574** backpack with a default address like `0x27` or `0x3F`





2. Software Setup:

- Change Time zone to match India: Start Preferences Raspberry Pi Configuration Localization Set Timezone Area: Asia, Location: Kolkata.
- Type the following commands in the terminal…
  - *pip3 install adafruit-circuitpython-charlcd*
- Open Thonny Python IDE and type the following code in the editor…

```
import time

from datetime import datetime

from RPLCD.i2c import CharLCD

# Initialize LCD

lcd = CharLCD('PCF8574', 0x27)

LCD_WIDTH = 16


def scroll_text(message, delay=0.3):

    """Scroll text from right to left."""

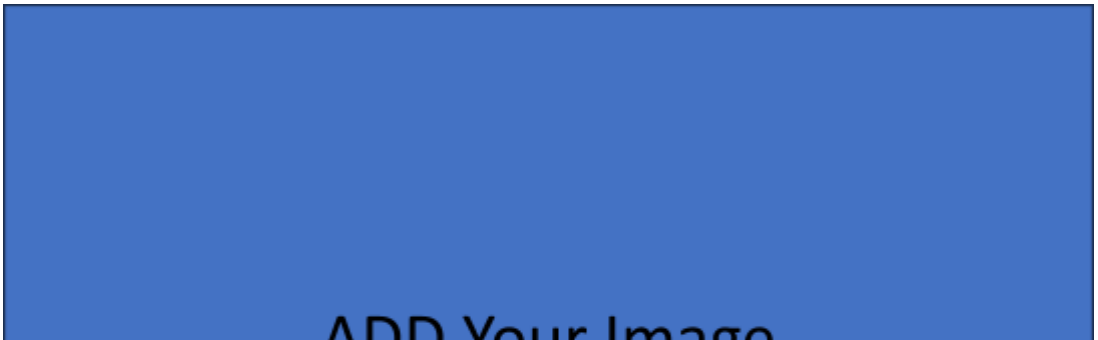
    text = " " * LCD_WIDTH + message + " " * LCD_WIDTH
```

```
    for i in range(len(text) - LCD_WIDTH + 1):
        lcd.clear()
        # Display scrolling text on first line
        lcd.cursor_pos = (0, 0)
        lcd.write_string(text[i:i+LCD_WIDTH])
        # Display current time on second line
        lcd.cursor_pos = (1, 0)
        current_time = datetime.now().strftime("%H:%M:%S")
        lcd.write_string(current_time)
        time.sleep(delay)
try:
    while True:
        # English Message
        scroll_text("Next Station Chembur", 0.25)


except KeyboardInterrupt:
    lcd.clear()
```

Precautions:

- If it is a fresh flash of Raspbian OS “*sudo apt update”* and “*sudo apt upgrade”* is a must thing.
- Connect the components before powering on the device and double check your connections.



Practical No: 04

Aim: Interface TM1637 with Raspberry Pi and Display current time.

Hardware Required:

1. Raspberry Pi 3B+
2. Ethernet Cable
3. Monitor
4. HDMI to VGI convertor
5. Micro SD card (any class best is class 10)
6. Adaptor with 5v 2A
7. USB mouse
8. USB keyboard
9. jumper wires
10. TM1637 (4 Digit 7 Segment Display)
11. Breadboard

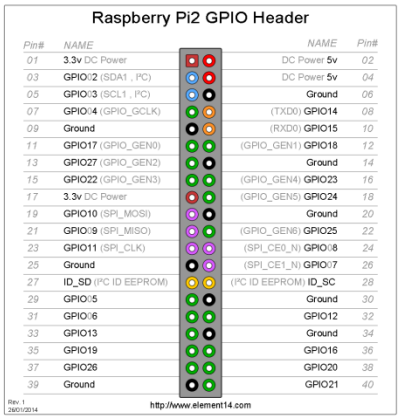
Software Required:

1. Rapbeian OS
2. Thonny Python IDE

Procedure:

1. **Hardware Setup:**
    - Connect the pins as given bellow….

    - **DIO → GPIO 2 (Pin 3)** on Pi
    - **CLK → GPIO 3 (Pin 5)** on Pi
    - **VCC → 5V (Pin 2 or 4)**
    - **GND → Ground (Pin 6)**



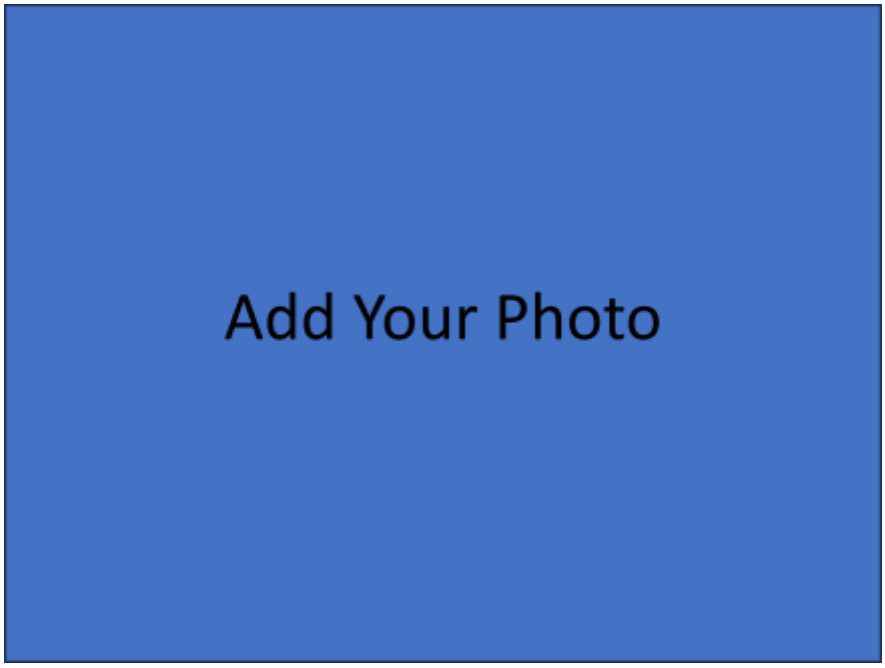


2. **Software Setup:**
    - Change Time zone to match India: Start Preferences Raspberry Pi Configuration Localization Set Timezone Area: Asia, Location: Kolkata.
    - Type the following commands in the terminal…
        - *pip3 install raspberrypi-tm1637 --break-system-packages*
    - Open Thonny Python IDE and type the following code in the editor…

```
import tm1637
from datetime import datetime
from time import sleep
tm = tm1637.TM1637(clk=3, dio=2)
while True:
  now = datetime.now()
  tm.numbers(now.minute,now.second)
  sleep(1)
```

Precautions:

- If it is a fresh flash of Raspbian OS “*sudo apt update”* and “*sudo apt upgrade”* is a must thing.
- Connect the components before powering on the device and double check your connections.



Practical No: 05

Aim: Interface RFID module with Raspberry Pi to read and write to RFID tags/cards.

Hardware Required:

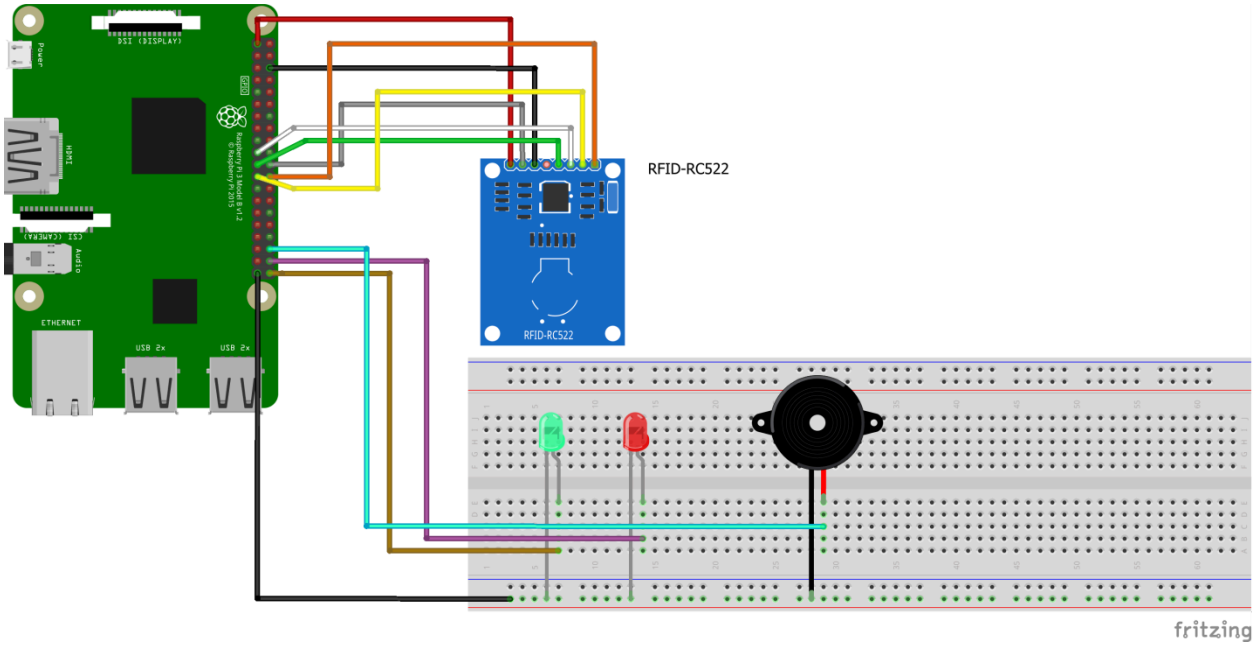
1. Raspberry Pi 3B+
2. 2.Ethernet Cable
3. Monitor
4. HDMI to VGA convertor
5. Micro SD card (any class best is class 10)
6. Adaptor with 5v 2A
7. USB mouse
8. USB keyboard
9. Relay board
10. Female – Female jumper wires. (7 numbers)
11. Female – Male jumper wires(3 numbers)
12. RFID-RC522
13. LED (red and green)
14. Bread board

Software Required:

1. Raspbian OS
2. Thonny Python IDE

Procedure:

1. **Hardware Setup:**
   - Connect as given below…
     - **SDA** connects to Pin **24**.
     - **SCK** connects to Pin **23**.
     - **MOSI** connects to Pin **19.**
     - **MISO** connects to Pin **21**.
     - **GND** connects to Pin **6**.
     - **RST** connects to Pin **22**.
     - **3.3v** connects to Pin **1**.
     - **GPIO20** connects to anode **LED red**
     - **GPIO21** connects to anode **LED green**
     - **GPIO16** connects to anode **Buzzer**



2. **Software Setup:**
   - Navigate to Start Preferences Raspberry Pi Configuration Interfaces Enable SPI and click on OK

- Reboot
- Open Terminal and type the following commands …
  - sudo apt update
  - sudo apt upgrade
  - sudo pip3 install mfrc522 –break-system-packages
- open Programming Thonny Python IDE New
- Type the following code in a new file to get the ID of the tag…

```
from mfrc522 import SimpleMFRC522
import RPi.GPIO as GPIO
import time

def main():
    # Initialize the reader
    reader = SimpleMFRC522()

    print("RFID Reader Ready")
    print("Hold a tag near the reader...")
    print("Press CTRL+C to exit.")

    try:
        while True:
            # Detect and read tag
            # read() returns a tuple: (id, text)
            id, text = reader.read()

            print("-" * 30)
            print(f"Tag Detected!")
            print(f"UID: {id}")
            # Decode text if it exists, otherwise show empty
            print(f"Data: {text.strip() if text else 'No text data'}")
            print("-" * 30)

            # Small delay to prevent reading the same tag multiple times instantly
            time.sleep(1)

    except KeyboardInterrupt:
        print("\nReader stopped by user.")
    finally:
        # Ensure GPIO pins are cleaned up
        GPIO.cleanup()

if __name__ == "__main__":
```

main()

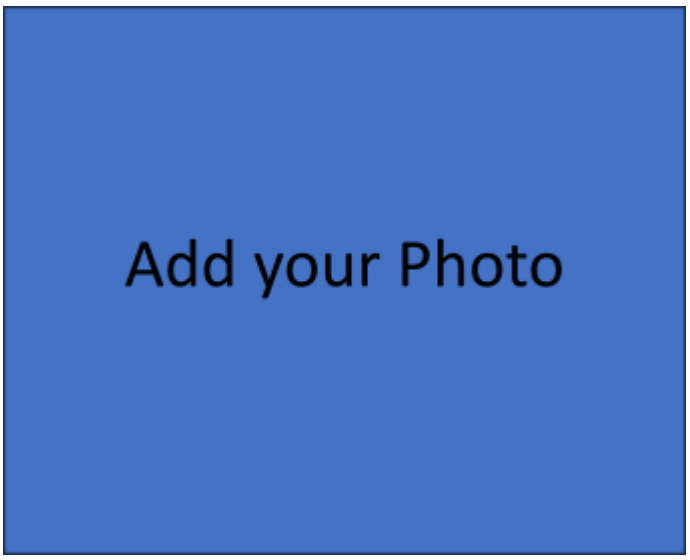
- Run the file by pressing play button in the top
- Place the tag near the RFID.
- Jot down the ID from the terminal
- Type the following code in a new file to write to tag…

Precautions:

- If it is a fresh flash of Raspbian OS "*sudo apt update"* and "*sudo apt upgrade"* is a must thing.
- Connect the components before powering on the device and double check your connections.
- When you are reading and writing to tag make sure you are in close proximity with the RFID.

Conclusion:

- We are able to authenticate a person on the basis of the physical tag a person is carrying.



Practical No: 6

Aim: Visitor Monitoring with Raspberry Pi and Pi Camera.

Hardware Required:

1. Raspberry Pi 3B+
2. Ethernet Cable
3. Monitor
4. HDMI to VGI convertor
5. Micro SD card (any class best is class 10)
6. Adaptor with 5v 2A
7. USB mouse
8. USB keyboard
9. Pi Camera

Software Required:

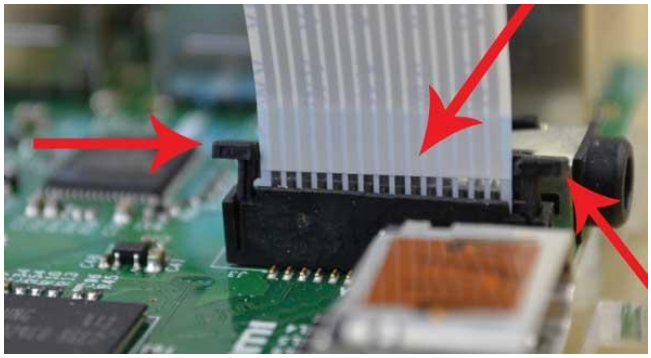
1. VIC media player

Procedure:

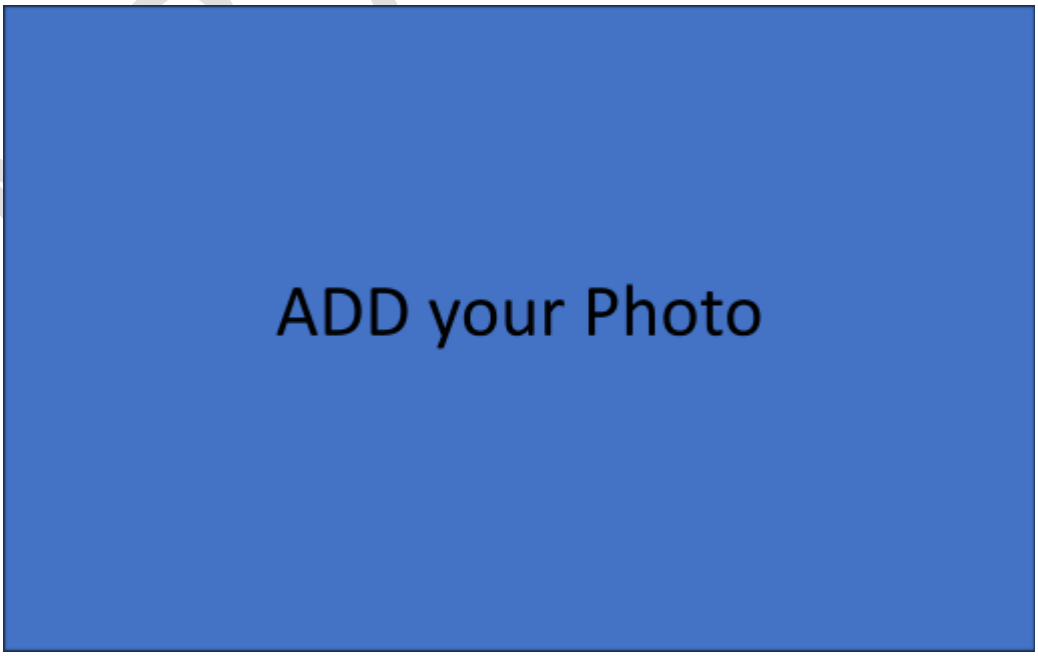
1. **Hardware Setup:**
    - Connect the Pi Camera to the camera port in the Raspberry pi.



MR ANUPAM



2. **Software Setup:**
   - Start Preferences Raspberry Pi Configuration Interfaces Enable Camera.
   - Install VLC in Raspberry pi by typing the following code in the terminal.
     - sudo apt install vlc
     - ifconfig (jot down the IP address of the raspberry Pi)
     - raspivid -o - -t 0 -n -w 600 -h 400 -fps 12 | cvlc -vvv stream:///dev/stdin --sout '#rtp{sdp=rtsp://:8554/}' :demux=h264
   - Install VLC In Desktop PC (https://www.videolan.org/vlc/download-windows.html)
   - Open VLC Media Player Media Open Network Stream type the following address in to the URL (rtsp://**<Your_IP_ADDRESS>:**8554/) use the jotted IP address.



ANUPAM

Practical No: 08

Aim: Raspberry Pi Module GPS Interfacing.

Hardware Required:

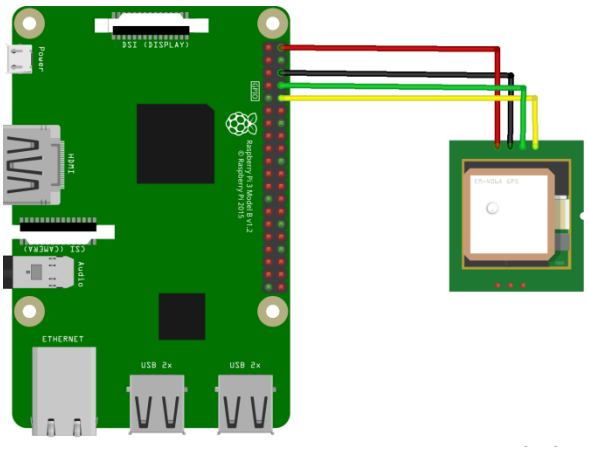
1. Raspberry Pi 3B+
2. Ethernet Cable
3. Monitor
4. HDMI to VGI convertor
5. Micro SD card (any class best is class 10)
6. Adaptor with 5v 2A
7. USB mouse
8. USB keyboard
9. Relay board
10. Female – Female jumper wires. (4 numbers)
11. GY-GPS6MV2

Software Required:

1. Rapbeian OS
2. Thonny Python IDE

Procedure:

1. **Hardware Setup:**
    - Connect the pins as given bellow….
        - **GPIO15** connects to Pin **Rx**.
        - **GPIO16** connects to Pin **Tx**.
        - **Pin6** connects to Pin **GND.**
        - **Pin2** connects to Pin **VCC**.



2. Software Setup:

- Navigate to Start Preferences Raspberry Pi Configuration Interfaces
  Enable Serial and click on OK .
- Type the following code in the terminal…
  - sudo pip install pynmea2
- Before typing the code we must ensure GPS is communicating with raspberry pi or not. Type the following code to check it works or not.
  - sudo cat /dev/ttyS0

```
pi@raspberrypi:~/Desktop/New $ sudo cat /dev/ttyS0
n msg*58
$GPTXT,01,01,01,NMEA unknown msg*58
$GPTXT,01,01,01,NMEA unknown msg*58
$GPTXT,01,01,01,NMEA unknown msg*58
$GPTXT,01,01,01,NMEA unknown msg*58
$GPTXT,01,01,01,NMEA unknown msg*58
$GPTXT,01,01,01,NMEA unknown msg*58
$GPTXT,01,01,01,NMEA unknown msg*58
$GPTXT,01,01,01,NMEA unknown msg*58
$GPRMC,174115.00,V,,,,,,,020619,,,N*76
$GPVTG,,,,,,,,,N*30
$GPGGA,174115.00,,,,,0,00,99.99,,,,,,*61
$GPGSA,A,1,,,,,,,,,,,,,99.99,99.99,99.99*30
```

- Open any editor you like and type the following python code

```
from time import sleep
from  serial import Serial
import string
import pynmea2
import webbrowser


port = "/dev/ttyS0"
ser = Serial(port, baudrate = 9600)
longitude_list=[]
latitude_list=[]
run=True
def longitude_average(longitudes):
    return sum(longitudes)/len(longitudes)
```

```
def latitude_average(latitudes):
   return sum(latitudes)/len(latitudes)

while run:
        try:
          data=ser.readline()
          if(data[0:6]=='$GPGGA'):
                msg=pynmea2.parse(data)
                lon=float(msg.lon)/100
                longitude_list.append(lon)
                lat=float(msg.lat)/100
                latitude_list.append(lat)

                strings="Lat: "+str(lat)+"  Lon: "+str(lon)
                print(strings)

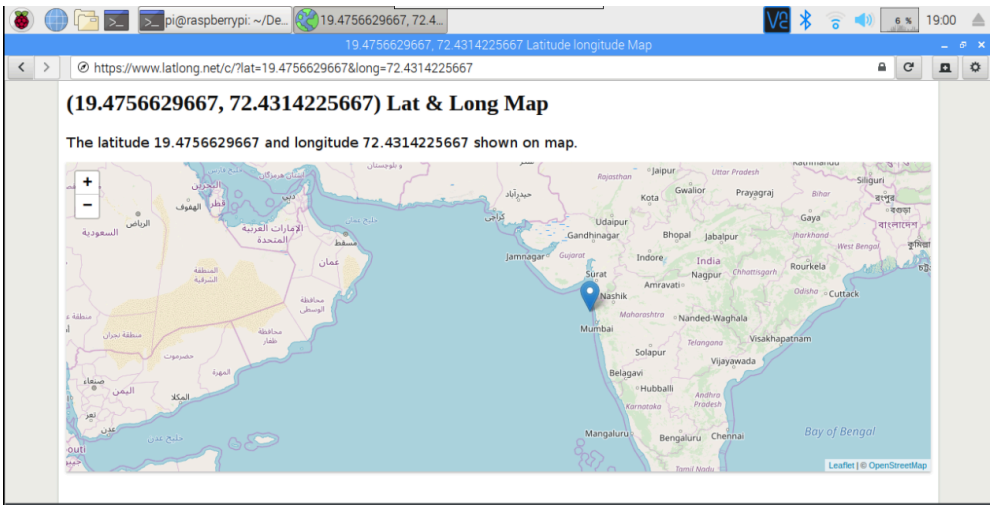
        except:
          averagedLog=longitude_average(longitude_list)
          averagedLat=latitude_average(latitude_list)
          print("Averaged Logitude: "+str(averagedLog)+" ""Averaged Latitude:
"+str(averagedLat))

webbrowser.open("https://www.latlong.net/c/?lat="+str(averagedLat)+"&long
="+str(averagedLog))
          run=False
```

- Open a terminal and navigate to the folder where you have save the python file and run the following command
  - sudo python **<filename>.**py

Precautions:

- If it is a fresh flash of Raspbian OS “*sudo apt update”* and “*sudo apt upgrade”* is a must thing.
- Connect the components before powering on the device and double check your connections.
- When you are reading and writing to tag make sure you are in close proximity with the RFID.


pi@raspberrypi: ~/De...
19.4756629667, 72.4...
19:00
19.4756629667, 72.4314225667 Latitude longitude Map
https://www.latlong.net/c/?lat=19.4756629667&long=72.4314225667
(19.4756629667, 72.4314225667) Lat & Long Map
The latitude 19.4756629667 and longitude 72.4314225667 shown on map.
Jaipur
Kota
Gwalior
Prayagraj
Udaipur
Gandhinagar
Bhopal
Jabalpur
Jamnagar
Indore
India
Surat
Nagpur
Amravati
Nashik
Mumbai
Nanded-Waghala
Solapur
Visakhapatnam
Vijayawada
Belagavi
Hubballi
Mangaluru
Bengaluru
Chennai
Gaya
Siliguri
Rourkela
Cuttack
Bay of Bengal
Leaflet | © OpenStreetMap

Practical No: 08

Aim: Design IoT based home automation system using Raspberry Pi.

Hardware Required:

1. Raspberry Pi 3B+
2. Ethernet Cable
3. Monitor
4. HDMI to VGA convertor
5. Micro SD card (any class best is class 10)
6. Adaptor with 5v 2A
7. USB mouse
8. USB keyboard
9. Relay board
10. Female – Female jumper wires.

Software Required:

1. Raspbian OS

Procedure:

1. **Hardware Setup:**
    - Connect according to the figure.

      GPIO20 □ IN1

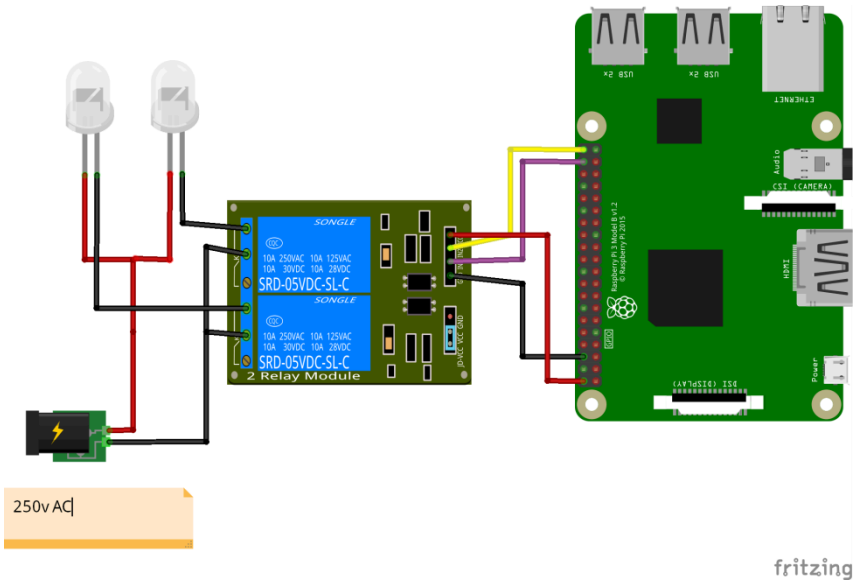
      GPIO21 □ IN2

      PIN2 □ VCC (5v)

      PIN6 □ GND

      *Black wire is the GND (ground)

Anupam



Raspberry Pi2 GPIO Header

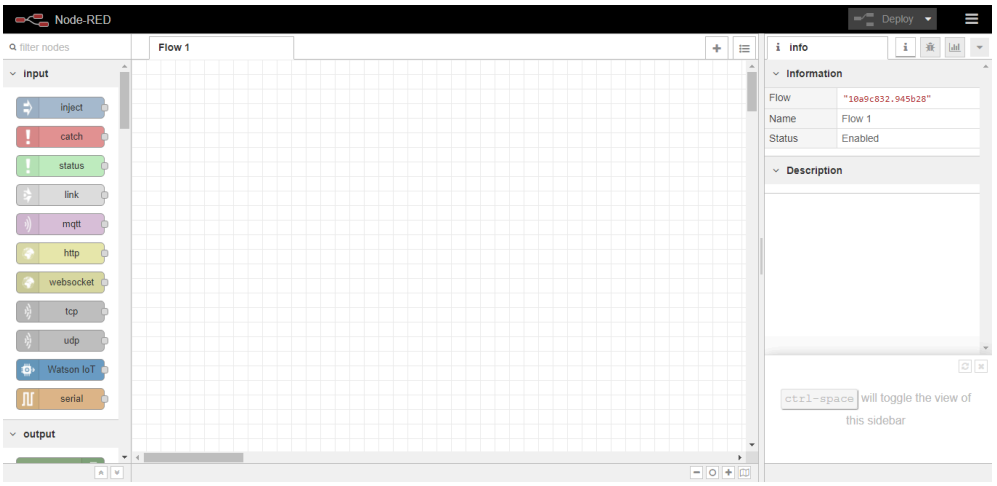
Pin#	NAME		NAME	Pin#
01	3.3v DC Power		DC Power 5v	02
03	GPIO02 (SDA1 , I²C)		DC Power 5v	04
05	GPIO03 (SCL1 , I²C)		Ground	06
07	GPIO04 (GPIO_GCLK)		(TXD0) GPIO14	08
09	Ground		(RXD0) GPIO15	10
11	GPIO17 (GPIO_GEN0)		(GPIO_GEN1) GPIO18	12
13	GPIO27 (GPIO_GEN2)		Ground	14
15	GPIO22 (GPIO_GEN3)		(GPIO_GEN4) GPIO23	16
17	3.3v DC Power		(GPIO_GEN5) GPIO24	18
19	GPIO10 (SPI_MOSI)		Ground	20
21	GPIO09 (SPI_MISO)		(GPIO_GEN6) GPIO25	22
23	GPIO11 (SPI_CLK)		(SPI_CE0_N) GPIO08	24
25	Ground		(SPI_CE1_N) GPIO07	26
27	ID_SD (I²C ID EEPROM)		(I²C ID EEPROM) ID_SC	28
29	GPIO05		Ground	30
31	GPIO06		GPIO12	32
33	GPIO13		Ground	34
35	GPIO19		GPIO16	36
37	GPIO26		GPIO20	38
39	Ground		GPIO21	40

Rev. 1
29/01/2014
http://www.element14.com

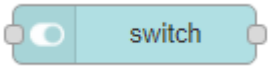
1. Software Setup:

- Open Terminal and type in the following commands
  - sudo apt install npm
  - update-nodejs-and-nodered
  - npm install node-red-dashboard
  - node-red-start

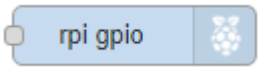
- Open browser and type the URL http://localhost:1880/



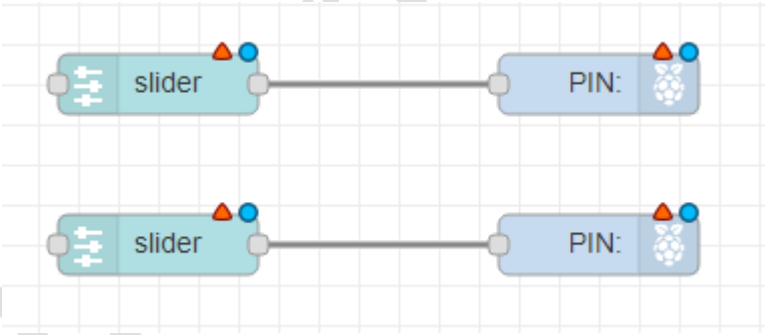
- Drag and drop 2 “switch” components from the left panel under dashboard category.



- Drag and drop 2 “rpi gpio” components from the left panel under Raspberry Pi category.



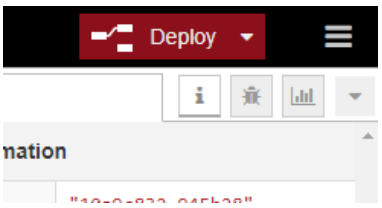
- Connect as shown in the figure.



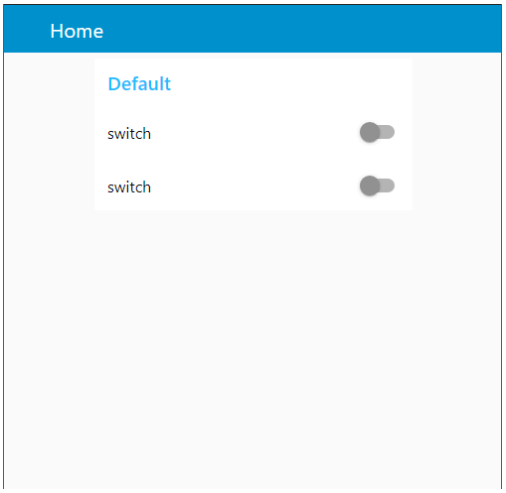
- Double click on any one component and create new Group by clicking the pencil icon.
- Click on the pencil icon next to Tab to add new Tab
- Click on “Add”
- Click on “Update”
- Click on “Done”
- Add the second switch to the same group which we have just created. ([Home] Default)
- Double Click on rpi gpio component and select GPIO20.
- Do the same with the other rpi gpio component but this time select GPIO21.
- Once you do the above steps correctly you will see no read triangle on the top.

Mr. Goupam

- Click on “Deploy” on top.



- Open http://localhost:1880/ui in new tab.



Precautions:

- If it is a fresh flash of “*Raspbian OS sudo”* apt update and “*sudo apt upgrade”* is a must thing.
- Connect the components before powering on the device and double check your connections.
- Take the help of the professor in connection as this project includes use of high voltage.

Conclusion:

- We are able to control two AC powered light bulbs using the web interface.

MR.Anupam

Practical No. 9

Aim

To simulate a fingerprint-based door locking system by interfacing a fingerprint sensor and a relay module with a Raspberry Pi, so that the door unlocks when an authorized fingerprint is detected.

Hardware Required

Sr. No.	Component	Quantity
1	Raspberry Pi 3 / Raspberry Pi 4	1
2	Fingerprint sensor (R307 / AS608 or compatible UART sensor)	1
3	5V relay module	1
4	Electronic door lock / solenoid lock (optional for physical demonstration)	1
5	External power supply suitable for the lock	1
6	Jumper wires	As required
7	Breadboard (optional)	1
8	USB cable and computer for programming	1

Software Required

1. Raspberry Pi OS.
2. Python 3.
3. Thonny IDE or any Python editor.
4. pyserial Python library.
5. adafruit-circuitpython-fingerprint library, if using a compatible sensor and library version.

Install the required libraries using the terminal:

```
python3 -m pip install pyserial adafruit-circuitpython-fingerprint
```

Mr. Anupam

Theory

A fingerprint-based door lock is a biometric security system that uses the unique patterns present on a person's fingertip to verify their identity.

The system works as follows:

1. The fingerprint sensor captures the fingerprint.
2. The sensor compares it with previously enrolled fingerprints.
3. If a matching fingerprint is found, the Raspberry Pi activates the relay.
4. The relay switches the door lock circuit to unlock the door.
5. If the fingerprint is not recognized, access is denied and the relay remains inactive.

The relay acts as an electrically controlled switch between the Raspberry Pi control circuit and the lock circuit.

Connections

The following is an example connection for a compatible UART fingerprint sensor and a 3.3V GPIO-compatible relay module.

A. Fingerprint Sensor to Raspberry Pi

Fingerprint Sensor Pin	Raspberry Pi Connection
VCC	Appropriate sensor supply, as specified by its datasheet
GND	GND
TX	GPIO15 / RXD, physical pin 10
RX	GPIO14 / TXD, physical pin 8

The UART connection uses crossed data lines: sensor TX to Pi RX, and sensor RX to Pi TX.

B. Relay Module to Raspberry Pi

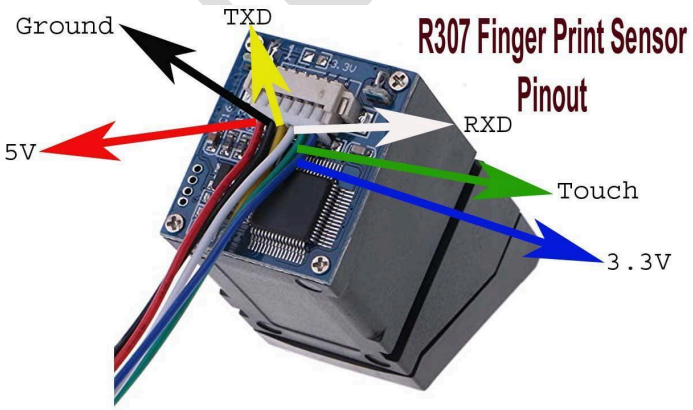
Relay Module Pin	Raspberry Pi Connection
VCC	Supply specified for the relay module
GND	GND
IN	GPIO17, physical pin 11

Important: Verify the relay's input voltage and trigger polarity before connecting it. Raspberry Pi GPIO pins use 3.3V logic and must never receive 5V directly. Do not power a solenoid lock from a GPIO pin; use an appropriately rated external supply and switching circuit.

Mr. Anupam

Procedure

1. Connect the fingerprint sensor to the Raspberry Pi using the UART interface.
2. Connect the relay module's control input to GPIO17.
3. Connect the relay's switching contacts to the separately powered lock circuit, if using a physical lock.
4. Install Raspberry Pi OS and the required Python libraries.
5. Enable the serial interface using Raspberry Pi configuration settings, and ensure the serial login console is disabled if it conflicts with the sensor connection.
6. Enrol an authorized fingerprint using the sensor's supported enrollment procedure.
7. Write and execute a Python program to read fingerprint matches and control the relay.
8. Place the enrolled finger on the sensor.
9. Observe whether the relay activates when the fingerprint matches.
10. Test with an unregistered fingerprint and verify that access is denied.
11. Record the observations and results in the journal.




```
import time
import serial
import RPi.GPIO as GPIO
import adafruit_fingerprint

# GPIO configuration
RELAY_PIN = 17

GPIO.setmode(GPIO.BCM)
GPIO.setup(RELAY_PIN, GPIO.OUT)

# Active-low relay: HIGH means relay OFF
GPIO.output(RELAY_PIN, GPIO.HIGH)

# Fingerprint sensor UART connection
uart = serial.Serial(
```

Mr. Anbam

```
    "/dev/serial0",
    baudrate=57600,
    timeout=1
)

finger = adafruit_fingerprint.Adafruit_Fingerprint(uart)


def unlock_door():
    print("Access Granted - Door Unlocked")

    GPIO.output(RELAY_PIN, GPIO.LOW)
    time.sleep(5)

    GPIO.output(RELAY_PIN, GPIO.HIGH)
    print("Door Locked")


try:
    if finger.verify_password():
        print("Fingerprint sensor connected")
    else:
        raise RuntimeError("Fingerprint sensor password check failed")

    print("Place your enrolled finger on the sensor")

    while True:
        result = finger.get_image()

        if result == adafruit_fingerprint.OK:
```

Mr. Anupam

```
            result = finger.image_2_tz(1)

            if result == adafruit_fingerprint.OK:
                result = finger.finger_search()

                if result == adafruit_fingerprint.OK:
                    print("Fingerprint ID:", finger.finger_id)
                    unlock_door()
                    time.sleep(1)

                else:
                    print("Access Denied - Fingerprint not recognized")

            else:
                print("Could not process fingerprint")

            # Wait for the finger to be removed
            while finger.get_image() == adafruit_fingerprint.OK:
                time.sleep(0.2)

        time.sleep(0.1)

except KeyboardInterrupt:
    print("\nProgram stopped")

finally:
    GPIO.output(RELAY_PIN, GPIO.HIGH)
    GPIO.cleanup()
    uart.close()
```

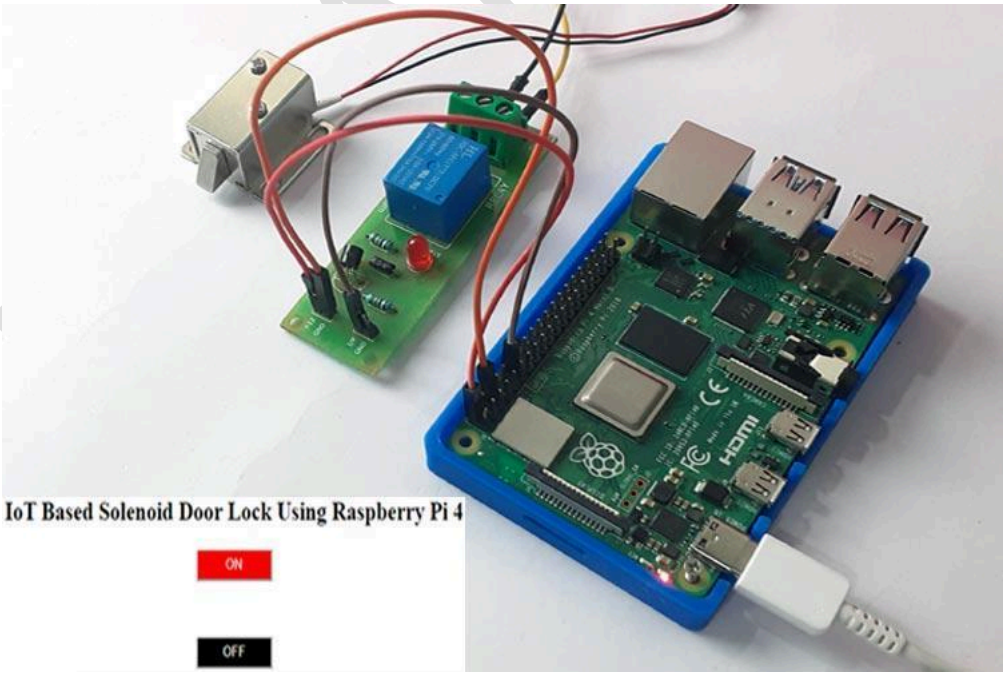
Mr. Anupam

Conclusion

Thus, a biometric door locking system was implemented using a Raspberry Pi and a fingerprint sensor. The practical demonstrates fingerprint authentication, GPIO control, relay interfacing, and the use of Python in an access-control application.

Precautions

1. Verify all pin connections before powering the circuit.
2. Do not apply 5V to Raspberry Pi GPIO pins.
3. Use a separate, correctly rated power supply for the electronic lock.
4. Test the relay without connecting the lock initially.
5. Ensure the fingerprint sensor is compatible with the selected Python library.
6. Keep the relay and lock wiring insulated to prevent short circuits.
7. For real doors, provide a safe emergency exit and a manual override; do not rely solely on the electronic lock.



Practical No. 10

Aim

To implement the Publisher–Subscriber communication model using the MQTT protocol with the Python Paho-MQTT library, where one Raspberry Pi publishes messages and another Raspberry Pi subscribes to the topic to control an LED.

Hardware Requirements

Component
Raspberry Pi 3 Model B/B+
Micro SD Card (16 GB or above)
LED (5 mm)
220 Ω Resistor
Breadboard
Jumper Wires
Power Adapter for Raspberry Pi
Wi-Fi Network / Ethernet

Software Requirements

Software
Raspberry Pi OS
Python 3
Paho-MQTT Library
Mosquitto Broker
GPIO Zero / RPi.GPIO
Thonny / VS Code

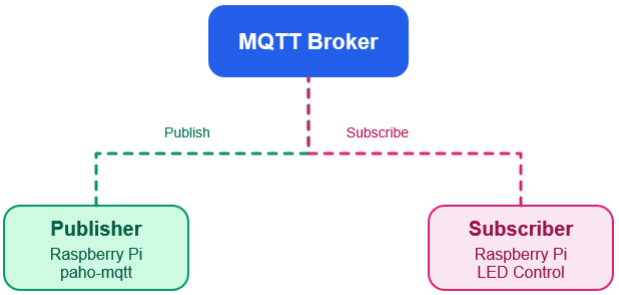
Theory

Mr. Anupam

What is MQTT?

MQTT (Message Queuing Telemetry Transport) is a lightweight messaging protocol designed for IoT devices. Instead of devices communicating directly, they exchange messages through a Broker.

MQTT Architecture

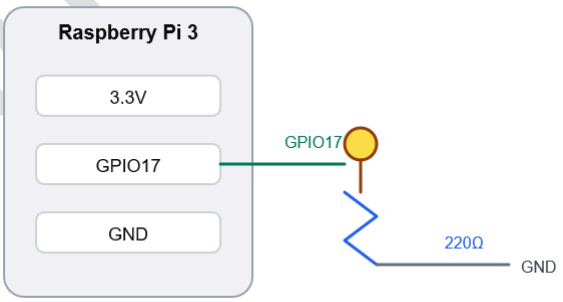


Working Principle

1. **The Publisher sends a message (ON or OFF) to a topic.**
2. **The MQTT Broker receives the message.**
3. **The Subscriber listens to the same topic.**
4. **According to the received message, the Raspberry Pi turns the LED ON or OFF.**

Circuit Diagram

LED connection with Raspberry Pi



GPIO Connection

Raspberry Pi Pin	Connected To
GPIO17 (Pin 11)	LED Anode (+)
LED Cathode	220 Ω Resistor
Resistor	GND (Pin 6)

Network Setup

All Raspberry Pi devices must be connected to the same Wi-Fi network.

Mr. Anupam

----------Subscriber.py----------

```
import paho.mqtt.client as mqtt
broker = "127.0.0.1"
topic = "home/led
client = mqtt.Client()
client.connect(broker,1883,60)
while True:
    msg = input("Enter ON or OFF : ")
    client.publish(topic,msg)
    print("Message Sent :",msg)
```

----------Publisher.py----------

```
import paho.mqtt.client as mqtt
from gpiozero import LED

led = LED(17)
topic = "home/led"
def on_message(client, userdata, message):
    msg = message.payload.decode()
    if msg == "ON":
        led.on()
        print("LED ON")
    elif msg == "OFF":
        led.off()
        print("LED OFF")
client = mqtt.Client()
```

Mr. Anupam

```
client.on_message = on_message
client.connect("127.0.0.1",1883,60)
client.subscribe(topic)
print("Waiting for Messages...")
client.loop_forever()
```

Installation Procedure

Step 1: Update Raspberry Pi

Open Terminal on both Raspberry Pis.

```
sudo apt update
sudo apt upgrade -y
```

Step 2: Install Mosquitto Broker

Install only on the broker Raspberry Pi.

```
sudo apt install mosquitto mosquitto-clients -y
```

Enable the service:

```
sudo systemctl enable mosquitto
sudo systemctl start mosquitto
```

Check status:

```
sudo systemctl status mosquitto
```

Step 3: Install Python MQTT Library

On both Raspberry Pis:

```
pip3 install paho-mqtt gpiozero
```

MQTT Topic

Parameter	Value
Broker IP	192.168.1.100 (Example)
Port	1883
Topic	home/led

Mr. Anupam

Messages ON, OFF

Replace the Broker IP with your Raspberry Pi broker's IP address.

Applications

- Smart Home Automation
- Remote Lighting Systems
- Industrial IoT Monitoring
- Smart Agriculture
- Wireless Sensor Networks
- Building Automation

Advantages of MQTT

- Lightweight protocol with low bandwidth usage.
- Fast real-time communication.
- Supports one-to-many communication.
- Reliable message delivery with QoS.
- Ideal for battery-powered IoT devices.

Precautions / Warnings

- Ensure all Raspberry Pi devices are connected to the same network.
- Verify the Broker IP address before running the programs.
- Connect the LED with a 220 Ω resistor to prevent damage.
- Do not connect the LED directly to 3.3V without a resistor.
- Start the Subscriber before the Publisher.
- Check GPIO pin numbers carefully (GPIO17, not physical pin number).

Result

The MQTT Publisher–Subscriber model was successfully implemented using the Paho-MQTT Python library. Messages published from one Raspberry Pi were received by the Subscriber through the Mosquitto Broker, and the LED was successfully controlled remotely using ON and OFF commands.

Mr. Anupam

Practical No. 11

Aim

To write and execute a Python program on Raspberry Pi to read and display the CPU utilization percentage, memory utilization percentage, and CPU temperature of the Raspberry Pi.

Hardware Required

Sr. No.	Hardware Component	Quantity
1	Raspberry Pi 3 / Raspberry Pi 4 / compatible model	1
2	MicroSD card with Raspberry Pi OS	1
3	Power supply for Raspberry Pi	1
4	Monitor, keyboard and mouse (optional when using SSH)	1 set
5	Internet connection (optional)	

Software Required

1. Raspberry Pi OS.
2. Python 3.
3. Thonny IDE or any Python editor.
4. psutil Python library.
5. Linux thermal-zone interface for reading CPU temperature.

Install the required library by executing the following command in the terminal:

6. `python3 -m pip install psutil`

Mr. Anupam

Theory

Raspberry Pi is a single-board computer that runs an operating system and performs various computing tasks. Monitoring its system resources helps users understand its performance and detect excessive CPU load, memory consumption, or overheating.

A. CPU Utilization (%)

CPU utilization indicates the percentage of CPU processing capacity being used by running processes during a time interval.

B. Memory Utilization (%)

Memory utilization indicates the percentage of available system RAM currently being used.

C. CPU Temperature (°C)

CPU temperature indicates the processor's current operating temperature in degrees Celsius. Monitoring temperature helps identify overheating and the need for better ventilation or cooling.

The Python library psutil provides CPU and memory statistics. CPU temperature is commonly obtained from a Linux thermal-zone interface on Raspberry Pi OS, although the available interface may vary by model and operating system.

Procedure

1. Start the Raspberry Pi and log in to Raspberry Pi OS.
2. Open the terminal or Thonny Python IDE.
3. Install the psutil library.
4. Create a new Python file named system_monitor.py.
5. Import the required Python libraries.
6. Write a program to read CPU usage, memory usage and CPU temperature.
7. Observe the displayed system statistics.
8. Record the observations in the practical journal.

```
import time

import psutil

from pathlib import Path
```

Mr. Anupam

```
def get_cpu_temperature():
    """Read CPU temperature from Linux thermal zones."""
    thermal_path = Path("/sys/class/thermal")

    for zone in sorted(thermal_path.glob("thermal_zone*")):
        try:
            temp_file = zone / "temp"

            if temp_file.exists():
                raw_temp = temp_file.read_text().strip()
                temperature = float(raw_temp)

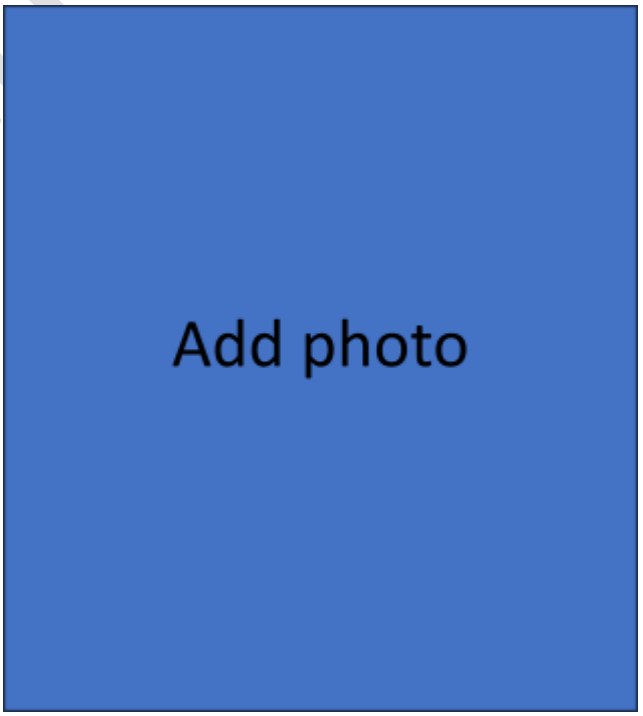
                # Linux thermal readings are usually in millidegrees C
                if temperature > 1000:
                    temperature /= 1000.0

                if 0 < temperature < 150:
                    return temperature

        except (OSError, ValueError):
            continue

    return None


try:
    print("RASPBERRY PI SYSTEM MONITOR")
    print("-" * 40)
```



Mr. Anupam

```
    # First call initializes CPU measurement
    psutil.cpu_percent(interval=None)

    while True:
        time.sleep(2)

        cpu_usage = psutil.cpu_percent(interval=None)
        memory = psutil.virtual_memory()
        cpu_temp = get_cpu_temperature()

        print("\nCPU Usage    : {:.2f}%".format(cpu_usage))
        print("Memory Usage : {:.2f}%".format(memory.percent))

        if cpu_temp is not None:
            print("CPU Temp     : {:.2f} °C".format(cpu_temp))
        else:
            print("CPU Temp     : Not available")

        print("-" * 40)

except KeyboardInterrupt:
    print("\nSystem monitoring stopped.")
```

Conclusion

Thus, the system monitoring program was implemented using Python on Raspberry Pi. The practical demonstrated how to access CPU performance, RAM usage, and processor temperature for monitoring the health and performance of a single-board computer.

Mr. Anupam